

Task 3: Secure Coding Review – CodeAlpha Internship

Prepared by: Iqra

Date: 10-Nov-2025

Executive Summary

During the security audit of the selected application (Python Flask web app), several critical and medium-severity vulnerabilities were identified. These include weak password storage, SQL injection risk, insecure file uploads, missing CSRF protection, and unsafe session management. Addressing these issues using secure coding practices and automated static analysis will significantly reduce the risk of exploitation and protect sensitive data.

The audit combines **manual code inspection**, **static analysis tools (Bandit, Semgrep, pip-audit)**, and **security best practices**, providing a roadmap for immediate and long-term remediation.

Findings & Recommendations

1. Weak Password Storage – High

Issue: Passwords are stored in plaintext or using fast, unsalted hashes (e.g., MD5/SHA1), making them vulnerable to compromise.

Risk: Account takeover in case of a database breach.

Recommendation:

- Use adaptive hashing algorithms like **bcrypt** or **Argon2**.
- Example with `werkzeug.security`:

```
from werkzeug.security import generate_password_hash,  
check_password_hash
```

```
pw_hash = generate_password_hash(password)
```

```
if check_password_hash(pw_hash, input_password):
```

```
    # Authentication successful
```

Enforce strong password policies (min length 12, mix of characters).

2. SQL Injection – High

Issue: Use of string concatenation in SQL queries exposes the app to injection attacks.

Risk: Unauthorized data access, database compromise.

Recommendation:

- Always use **parameterized queries** or ORM methods.

Unsafe

```
cursor.execute(f'SELECT * FROM users WHERE email={email}')
```

Safe

```
cursor.execute("SELECT * FROM users WHERE email = ?", (email,))
```

3. Insecure File Upload – High

Issue: Users can upload files without validation or safe filename handling.

Risk: Remote code execution, XSS, file overwrite.

Recommendation:

- Validate file type (extension + MIME).
- Use `secure_filename()` and store files outside the webroot.
- Limit file size and only allow safe types (e.g., images).

```
from werkzeug.utils import secure_filename
import uuid, os
filename = secure_filename(file.filename)
filename = f'{uuid.uuid4().hex}_{filename}'
file.save(os.path.join(UPLOAD_DIR, filename))
```

4. Missing CSRF Protection – High

Issue: POST endpoints lack CSRF tokens.

Risk: Attackers can perform unauthorized actions on behalf of users.

Recommendation:

- Implement **Flask-WTF CSRF** or Flask-SeaSurf.
- For APIs, require anti-CSRF headers and SameSite cookies.

5. Session Cookie Misconfiguration – Medium/High

Issue: Cookies missing Secure, HttpOnly, SameSite flags; weak SECRET_KEY.

Risk: Cookie theft, session hijacking.

Recommendation:

```
app.config.update(
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE='Lax',
    SECRET_KEY=os.environ['SECRET_KEY']
)
```

6. Missing Security HTTP Headers – Medium

Issue: CSP, X-Frame-Options, X-Content-Type-Options not set.

Risk: XSS, clickjacking, MIME-type attacks.

Recommendation:

- Use **Flask-Talisman** to enforce secure headers.

```
from flask_talisman import Talisman
Talisman(app, content_security_policy={"default-src": ["'self'"]})
```

7. Unsafe eval/exec/pickle Usage – High

Issue: Executing untrusted input or deserializing untrusted data.

Risk: Remote code execution.

Recommendation:

- Avoid eval and exec.
- Use **JSON** or **msgpack** for serialization instead of pickle.

8. Insufficient Input Validation & Output Encoding – Medium

Issue: User input not validated or escaped, enabling XSS.

Recommendation:

- Always validate input length, type, and range.
- Use **auto-escaping templates** in Jinja.

9. Dependency Vulnerabilities – Medium

Issue: Outdated third-party libraries.

Recommendation:

- Use pip-audit, safety, or **Dependabot** to detect vulnerabilities.
- Keep dependencies up-to-date with pinned versions.

10. Logging Sensitive Data – Low/Medium

Issue: Logging passwords or tokens in plaintext.

Recommendation:

- Never log sensitive data.
- Use structured logging with redaction.

Secure Coding Best Practices

- Principle of **least privilege**: limit database and system access.
- Always validate input and encode output.
- Fail securely; avoid exposing system details in errors.
- Enforce **HTTPS** and secure cookies.
- Manage secrets via environment variables or secret managers.
- Maintain dependency hygiene with regular scanning.
- Log safely; avoid storing PII.
- Conduct threat modeling and secure design.
- Integrate security checks in code reviews.

CI/CD & QA Recommendations

- Pre-merge checks: **Bandit**, **Semgrep**, **pip-audit**.
- Use **detect-secrets** pre-commit hooks.
- Automated tests for authentication, file upload, and validation.
- Daily dependency scanning via **Dependabot** or CI pipelines.
- Integrate SAST tools in PR workflows.

Conclusion

The audit identified critical and medium-risk vulnerabilities that can compromise user data and system integrity. Prioritizing password security, SQL injection fixes, CSRF protection, and file upload safety will significantly improve application security.

Implementing the recommended practices, automated scanning, and secure coding standards ensures a robust, safe, and maintainable codebase, ready for deployment in a production environment.
